Solutions

1. Linked list reverse function

SOLUTION:

```
void reverse()
{
    node* prev = NULL;
    node* current = head;                                    //      [1]
    node* next = NULL;
    while (current != NULL) {
        next = current->next;  // Store next                        [2]
        current->next = prev;  // Reverse current node's pointer    [2]
        prev = current;        // Move pointers one position ahead. [2]
        current = next;        //                                   [1]
    }
    head = prev;
}
```

//note: marks will be awarded if the logic is in the right direction and not otherwise. Alternate solutions will receive credit as well.

2. Pattern question

```
{
  if(i==1||i==n)
            (j<n-j)?printf("%d",j):printf("%d", n-j+1);          //2m
  else if (j==1||j==n)
           (i>n-i)?printf("%d",i):printf("%d", n-i+1);           //2m
 else if(j==n-i+1||i==j)
           (j%2)?printf("%d",j-2):printf("%d",j+2);             //2m
else
       printf("~");                                             //2m
}
```

//note: if your solution uses more than one line (if..else..) then the max marks for that blank are 1. Partial marks will be given if, in any blank, your logic is partially correct and in the right direction.

Design question on rational numbers:

No extra marks for writing the whole driver/working code with the main function

a. *3M - Float as a possible data type is totally wrong. 0 marks for float.*

```
typedef struct rational rat;
struct rational {
      int p, q;
};
```

b. *2M - 0 marks if not writing function prototypes/declarations*

```
rat add(rat ratnum1, rat ratnum2);
rat reduce(rat ratnum);
```

c. 4M

```
rat add(rat ratnum1, rat ratnum2) {

      int u, v, w, x, y, z;
       w = ratnum1.p;
       x = ratnum1.q;
       y = ratnum2.p;
       z = ratnum2.q;
       u = w*z + x*y;
       v = x*z;
       rat ratnum3 = {u, v};
       return ratnum3;

}

 rat reduce(rat ratnum) {

       int a = ratnum.p, b = ratnum.q;
       rat rat_reduce;
       rat_reduce.p = a/gcd(a,b);
       rat_reduce.q = b/gcd(a,b);
       return rat;

}
```

4. a) This function prints the content of every file passed in the command line arguments from argv[1] onwards.

b) Equivalent command: “cat file1 file2…”

5)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 50

int isDictEntry(char *, char *[], int);

int main()
{
  char w1[MAX], w2[MAX], combined[MAX];
```

```
  int i,j,N;
  scanf("%d",&N);
  char *words[N];

  for (i=0; i<N; ++i)
  {
   scanf("%s",w1);
   if (strlen(w1) == 1) continue;
   words[i] = (char *) malloc(strlen(w1)+1);
   strcpy(words[i],w1);
  }
  for (i=0; i<N; ++i)
    for (j=0; j<N; ++j)
    {
      strcpy(combined,words[i]);
      strcat(combined,words[j]);
      if (isDictEntry(combined,words,count) == 1)
        printf("%s + %s = %s\n", words[i], words[j], combined);
      strcpy(combined,"");
    }
  return 0;
  for (i=0; i<N; ++i)
    free(words[i]);
  return 0;
}

int isDictEntry(char *w, char *list[], int n)
{
  int i;
  for (i=0; i<n; ++i)
    if (strcasecmp(w,list[i]) == 0)
        return 1;
  return 0;
}
```